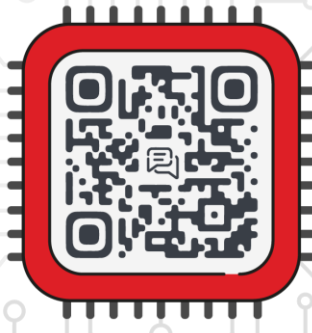




عملي

4

19

2850 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

تحليل النقاط الوظيفية

(Function Point Analysis)

عملي مشترك



RB Informatics; 05/05/2026

محتوى مجاني غير مخصص للبيع التجاري

هندسة برمجيات ٢

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

مرحباً يا أصدقائي! بعد أن تعرفنا في المحاضرات السابقة على أدوات وتقنيات تساعدنا في تخطيط المشروع البرمجي وإدارته وتنظيمه، بدءاً من WBS (Work Breakdown Structure) لتقسيم المشروع إلى مهام صغيرة ومنظمة، ثم أدوات الإدارة العملية مثل Jira و Microsoft Project لمتابعة المهام والفرق الزمنية، وانتقلنا بعدها إلى مخططات إدارة المشروع مثل Network diagram لتحليل العلاقات بين الأنشطة، و Gantt chart لتمثيل الجدول الزمني للمشروع بصورة بصرية واضحة.

نتنقل في هذه المحاضرة إلى جانب آخر أكثر أهمية في هندسة البرمجيات، وهو تقدير حجم النظام البرمجي وقياسه بطريقة علمية قبل البدء بالتنفيذ الفعلي.

حيث سنتعرف على مفهوم Function Point Analysis (FPA) أو تحليل النقاط الوظيفية، وهي تقنية تستخدم لقياس حجم البرمجيات اعتماداً على الوظائف والخدمات التي يقدمها النظام للمستخدم، بدلاً من الاعتماد فقط على عدد الأسطر البرمجية.

وتعد هذه الطريقة من الأساليب المهمة في تقدير:

- حجم المشروع.
- الجهد المطلوب للتطوير.
- الزمن والتكلفة.
- إنتاجية الفريق البرمجي.

كما سنفهم كيف يساعد تحليل النقاط الوظيفية في إعطاء تقديرات أكثر واقعية للمشاريع البرمجية، خاصة في المراحل الأولى من التحليل والتخطيط.



أولاً: ما هو الـ FP:

هو أسلوب لقياس حجم البرمجيات من منظور المستخدم، أي بناءً على ما يقدمه التطبيق من وظائف للمستخدم، وليس بناءً على عدد أسطر الكود.

التكلفة :

ذكرنا في المقدمة أن هذه الطريقة تساعدنا في تقدير التكلفة:

كلما زادت الوظائف الموجودة في النظام، زاد عدد الـ FP الموجودة في النظام، زاد الجهد والزمن، بالتالي ارتفع سعر المشروع.

ثانياً: مكونات الـ FP الأساسية:

تعتمد الـ FPA على 5 محاور أساسية:

5

(ELF)
External
Interface Files:
الملفات الخارجية.

4

(ILF)
Internal Logical
Files:
الملفات الداخلية.

3

(EQ)
External
Inquiries:
الاستعلامات.

2

(EO)
External
Outputs:
المخرجات والتقارير.

1

(EI)
External
Inputs:
مدخلات المستخدم.

بنظرة عامة:

ما العمليات التي يدخل فيها المستخدم بيانات للنظام؟

مثلاً:

- Login.
- إضافة منتج.
- رفع ملف.
- تعديل بيانات.

هذه EI.

ما هي البيانات التي يرسلها النظام للمستخدم؟

مثلاً:

- تقارير.
- فواتير.
- ملفات pdf.

هذه EO.

ما هي الاستعلامات؟

هي طلبات عرض البيانات بدون تعديل عليها وبدون معالجة معقدة.

مثلاً:

- البحث عن مستخدم.
- عرض كشف الحساب.
- فلترة المنتجات.

هذه EQ.

ما هي البيانات التي يخزنها النظام ويتحكم بها؟

مثلاً:

- جدول المستخدمين
- المنتجات
- الطلبات

هذه ILF.

ما هي الملفات أو البيانات القادمة من خارج النظام؟

مثلاً:

- نظام دفع خارجي.
- قاعدة بيانات خارجية.
- API خارجي.

هذه ELF.

إضاءة:

الفرق بين الـ EQ والـ EO :

الفرق الأساسي هو كمية المعالجة التي يقوم بها النظام على البيانات

حيث EO مخرجاته فيها معالجة أو حسابات أو تنسيق معقد

بينما الـ EQ استعلامات بسيطة لعرض بيانات موجودة بدون معالجة كبيرة .



ثالثاً: حساب الوزن:

حسب تلك المحاور، نريد أن نعرف:

1. يمكننا تجميعهم في نوعين من المكونات:

مكونات البيانات (data functions) : ILF, ELF .

مكونات المعاملات (transaction functions) : EI, EO, EQ .

2. مستويات التعقيد Complexity Levels:

ليس كل input يساوي الآخر، فبعض العمليات بسيطة وبعضها معقد،

نقسمهم إلى :

- High
- Average
- Low

ويكون مستوى التعقيد هذا على حسب مفاهيم:

1. Data Element Type (DET):

عدد الحقول أو البيانات الفريدة التي يتعامل معها النظام
أي : كم عنصر بيانات يراه المستخدم، أو يدخله أو يسترجعه.

المثال الأول:

"شاشة تسجيل مستخدم" فيها الحقول: name, email, password, phone
إذاً DET=4.

المثال الثاني:

"الفاتورة" فيها الحقول: ID, اسم العميل, السعر الإجمالي, الضريبة, التاريخ, طريقة الدفع.
إذاً DET=6
هنا DET أكبر يعني التعقيد أكبر.

2. (RET) Record Element Type

يستخدم مع ILF, ELF, وهو عدد المجموعات المنطقية للبيانات داخل الملف، يعني كم قسم أو كيان منطقي موجود.
المثال الأول:

"نظام الجامعة" يحتوي: students, courses, المدرسين
وهذه مجموعات منطقية مختلفة، بالتالي RET=3

المثال الثاني:

جدول Users فقط مكون من: اسم و إيميل و كلمة مرور
هنا الـ RET=1، لأنه مجموعة منطقية واحدة فقط

3. (FTR) File Type Referenced

يستخدم مع EI, EO, EQ وهو : كم ملف أو مصدر بيانات تعتمد عليه العملية؟
مثال:

"عملية إنشاء طلب" تتعامل مع: User Table, Products Table, Orders Table
إذاً: FTR=3.

ملاحظة:

لا يمكن تحديد التعقيد بشكل عشوائي بل نقوم أولاً بحساب DET ثم نحسب FTR أو RET.

3. جدول الأوزان:

Low	Average	High	النوع
3	4	6	EI
4	5	7	EO
3	4	6	EQ
7	10	15	ILF
5	7	10	EIF

4. جداول التعقيد:

قلنا أن:

- RET يستخدم مع ILF، ELF.
- FTR يستخدم مع EQ، EI، EO.

إذاً، لدينا جدولي تعقيد:

1. جدول تعقيد EIF / ILF:

- يستخدم مع EIF، ILF .
- ويعتمد على DET، RET .

DET +51	DET 20-50	DET 1-19	
Average	Low	Low	RET 1
High	Average	Low	RET 2-5
High	High	Average	RET +6

كيف نقرأ الجدول؟

مثلاً لدينا :

RET=3

DET=40

نبحث عن الصف : RET 2-5

العمود: DET 20-50:

2. جدول تعقيد EI:

- يستخدم مع EI.
- ويعتمد على DET، FTR.

DET +16	DET 5-15	DET 1-4	
Average	Low	Low	FTR 0-1
High	Average	Low	FTR 2
High	High	Average	FTR +3

3. جدول تعقيد EO:

- يستخدم مع EO.
- ويعتمد على DET، FTR.

DET +20	DET 6-16	DET 1-5	
Average	Low	Low	FTR 0-1
High	Average	Low	FTR 2-3
High	High	Average	FTR +4

4. جدول تعقيد EQ:

يتم حسابها بطريقة خاصة، لأنها عبارة عن جانب إدخال وإخراج ينتج عنها استرجاع بيانات فقط:

- نحسب جانب الإدخال من جدول EI.
- ونحسب جانب الإخراج من جدول EO.
- ونأخذ الأعلى منهما

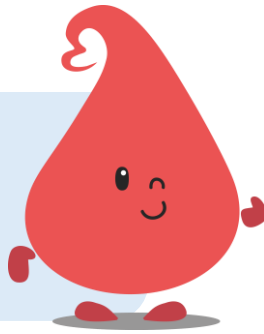
مثال :

الإدخال : Average

الإخراج : High

إذاً: EQ = High

"All You Need Is To
Believe In Yourself."



بعد التعرف على الجداول، يصبح التسلسل الكامل كالتالي:

عند EIF, ILF:

- نتعامل مع DET, RET.
- نذهب لجداول EIF, ILF.
- نحصل على ناتج Low/ Average/ High.
- نذهب لجداول الأوزان.
- نحصل على النقاط.

عند EI, EO, EQ:

- نتعامل مع DET, FTR.
- نذهب لجداول EO أو EI.
- نحصل على ناتج Low/ Average/ High.
- نذهب لجداول الأوزان.
- نحصل على النقاط.

مثال كامل:

عملية "إضافة طلب":

1. نوع العملية:

نرى أنها عملية Input إذاً EI.

2. نحدد FTR و DET :

تتعامل مع : Order Table ،Products Tabel ،User Table

إذاً $FTR = 3$.

ونفترض أن الشاشة تحتوي 18 حقلاً

$DET = 18$.

3. نذهب لجدول EI :

الصف: $FTR + 3$

العمود : $DET + 16$

النتائج أن هذه العملية تعقيدها مرتفع **High**.

4. من جدول الأوزان، عند EI و High :

يكون الوزن = 6 وانتهى حساب هذا العنصر.

والآن بعد أن صنفنا الوظائف وأعطينا وزن لكل منها ، نقوم بحساب حجم النظام.

رابعاً: Unadjusted Function Points (UFP) :

النقاط الوظيفية غير المعدلة، أو "الحجم الخام للنظام"

سميت هكذا لأننا حتى الآن حسبنا الوظائف فقط، ولم نأخذ بعين الاعتبار: الأداء، التوزيع، التعقيد التقني، الأمان، تعدد المستخدمين، البيئة التشغيلية.

أي: ما زلنا في مرحلة حساب الحجم الوظيفي الأساسي فقط :

■ الفكرة الرياضية:

بعد أن أصبح لدينا الوزن، التعقيد ، النوع نقوم بجمع جميع الأوزان حسب النوع والتعقيد.

■ المعادلة :

$$\sum count \times weight$$

■ Count: هو عدد الوظائف من نفس النوع والتعقيد.

مثال: لدينا 3 عمليات من النوع EI بتعقيد Average إذاً $\text{count} = 3$.

▪ Weight: الوزن المأخوذ من جدول الأوزان.

مثلاً: EI average = 4 إذاً ال weight = 4.

لدينا 3 عمليات EI كل واحدة وزنها 4 إذاً $12 = 4 \times 3$

وهذه مساهمة نوع ال EI وحده داخل ال UFP

لذا نكرر العملية لكل الأنواع EI، EO، EQ، ILF، EIF

ثم نجمعهم جميعاً.

فالصيغة التفصيلية :

$$UFP = (EI \times Weight) + (EO \times Weight) + (EQ \times Weight) + (ILF \times Weight) + (EIF \times Weight)$$

لكن عملياً، كل مستوى تعقيد (High، Average، Low) له وزن مختلف ، لذا بشكل أدق:

$$UFP = \sum (EI_L + EI_A + EI_H + EO_L + EO_A + EO_H + \dots)$$

والآن:

سؤال: هل نظامان يمتلكان نفس عدد الوظائف، يمتلكان نفس مستوى الصعوبة دائماً؟

نظامان كلاهما $UFP = 76$

لكن !

النظام الأول هو Blog بسيط وله مستخدمون قليلون.

والنظام الثاني هو Banking system أو آلاف المستخدمين، الأمان عالٍ و real time

هل هما متساويان؟ طبعاً لا

ومن هنا ظهرت فكرة: TDI / RCAF



خامساً: التعقيد التقني TDI/RCAF:

(RCAF) Relative Complexity Adjustment Factor تسمى (TDI) Total Degree of Influence

وهو مجموع تأثير الخصائص التقنية للنظام.

1. كيف يحسب؟

لدينا 14 عاملاً تقنياً، وكل عامل يأخذ قيمة من 0-5. نجمعهم $TDI = \sum DI_i$ كي نحسب عامل تعديل الحجم (VAF) Value Adjustment Factor

$$VAF = 0.65 + (0.01 \times TDI)$$

المعادلة النهائية:

$$FP = UFP \times VAF$$

أو

$$FP = UFP \times (0.65 + 0.01 \times TDI)$$

2. العوامل الـ 14:

قلنا أن لدينا 14 عاملاً كل منهم يأخذ قيمة من 0-5.
 ▪ معنى القيم:

0 1 2 3 4 5

تأثير شديد جداً تأثير كبير تأثير متوسط تأثير متوسط قليلاً تأثير بسيط لا يوجد تأثير

▪ العوامل:

1. اتصالات البيانات (Data Communications):

- هل النظام يرسل بيانات عبر الشبكة؟
- يتواصل مع أنظمة أخرى؟
- يستخدم بروتوكولات اتصال؟

كلما زاد التواصل زاد التعقيد.

الوصف	الدرجة
تطبيق دفعي (Batch) أو حاسب مستقل (Standalone PC).	0
تطبيق دفعي مع إدخال أو طباعة عن بُعد.	1
تطبيق دفعي مع إدخال و طباعة عن بُعد.	2
جمع بيانات Online أو واجهة TP (Teleprocessing) لعملية دفعية أو نظام استعلام.	3
أكثر من واجهة، لكن يدعم نوعاً واحداً من بروتوكولات TP.	4
يدعم أكثر من نوع من بروتوكولات TP.	5

2. المعالجة الموزعة (Distributed Data Processing) :

- هل المعالجة موزعة على عدة servers؟
- عدة nodes؟
- عدة services؟

الوصف	الدرجة
لا يدعم نقل البيانات أو الوظائف بين مكونات النظام.	0
يجهز البيانات لمعالجتها على مكون آخر (مثل Excel أو DBMS على PC).	1
البيانات تجهز ثم تنقل وتعالج على مكون آخر (ليس للمستخدم النهائي).	2
معالجة موزعة ونقل بيانات Online باتجاه واحد فقط.	3
معالجة موزعة Online باتجاهين.	4
الوظائف تنفذ ديناميكياً على المكون الأنسب من النظام.	5

3. الأداء (Performance) :

- هل يوجد قيود زمنية؟ real time ؟

الوصف	الدرجة
لا يوجد متطلبات أداء خاصة.	0
متطلبات أداء وتصميم مذكورة لكن بدون إجراءات خاصة.	1
زمن استجابة حرج في ساعات الذروة، الموعد النهائي اليوم التالي.	2
زمن استجابة حرج في كل ساعات العمل + قيود على الأنظمة المتفاعلة.	3
بالإضافة لما سبق: مهام تحليل أداء في مرحلة التصميم.	4
استخدام أدوات تحليل أداء في التصميم/التطوير/التنفيذ.	5

4. التشغيل المكثف للموارد (Heavily Used Configuration):

- التشغيل المكثف للموارد، هل النظام يعمل تحت ضغط عالٍ؟

الدرجة	الوصف
0	لا قيود تشغيلية.
1	قيود موجودة لكنها أقل من المعتاد، لا حاجة لجهد خاص.
2	اعتبارات أمنية أو زمنية.
3	متطلبات معالج محدد لجزء من التطبيق.
4	قيود خاصة على المعالج المركزي أو معالج مخصص.
5	قيود إضافية على المكونات الموزعة للنظام.

5. معدل المعاملات (Transaction Rate):

- كم عدد العمليات التي تحدث؟
مثلاً: آلاف الطلبات بالدقيقة.

الدرجة	الوصف
0	لا توجد فترة ذروة متوقعة.
1	ذروة شهرية/فصلية/موسمية/سنوية.
2	ذروة أسبوعية.
3	ذروة يومية.
4	معدل معاملات عالٍ يستلزم تحليل أداء في التصميم.
5	بالإضافة لما سبق: استخدام أدوات تحليل أداء في كل المراحل.

6. الإدخال المباشر (Online Data Entry):

- الإدخال المباشر.
هل المستخدمون يدخلون البيانات online مباشرة؟

الدرجة	نسبة المعاملات التفاعلية
0	كل المعاملات Batch
1	1% - 7%
2	8% - 15%
3	16% - 23%
4	24% - 30%
5	أكثر من 30%

7. كفاءة المستخدم النهائي (End-User Efficiency):

- كفاءة المستخدم النهائي.
- هل النظام يحتاج UX متقدم؟ سرعة استخدام؟ واجهات ذكية واختصارات؟ تشمل ميزات مثل:

- مفاتيح وظيفية وقوائم ديناميكية.
- مساعدة Online ووثائق.
- تحريك المؤشر تلقائياً، Scrolling.
- طباعة عن بُعد، نوافذ منبثقة، اختيار من الشاشة بالماوس.
- ألوان وتظليل واستخدام Reverse Video.
- دعم لغتين (يُحسب 4 بنود) أو متعدد اللغات (يُحسب 6 بنود).

الدرجة	الوصف
0	لا شيء مما سبق.
1	1 إلى 3 من البنود.
2	4 إلى 5 من البنود.
3	6 فأكثر بدون متطلبات كفاءة محدّدة.
4	6 فأكثر مع متطلبات تستوجب مهام تصميم Human Factors.
5	6 فأكثر مع متطلبات تستوجب أدوات وعمليات خاصة لإثبات تحقق الأهداف.

8. التحديث المباشر (Online Update):

- هل البيانات يتم تحديثها مباشرة أثناء التشغيل؟

الدرجة	الوصف
0	لا يوجد.
1	تحديث Online لـ 1-3 ملفات تحكم، حجم منخفض، استرجاع سهل.
2	تحديث Online لـ 4 فأكثر، حجم منخفض، استرجاع سهل.
3	تحديث Online لملفات ILF رئيسية.
4	بالإضافة: حماية من فقد البيانات مصممة بشكل خاص.
5	بالإضافة: أحجام كبيرة + إجراءات استرجاع آلية بأقل تدخل بشري.

9. المعالجة المعقدة (Complex Processing):

العناصر الخمسة:

- تحكم حسّاس (Audit) أو معالجة أمنية خاصة.
- معالجة منطقية واسعة.
- معالجة رياضية واسعة.
- معالجة استثناءات كثيرة (معاملات غير مكتملة).
- معالجة معقدة لتعدد مدخلات/مخرجات.

(Multimedia)، (Device independence)

الدرجة	عدد البنود
0	لا شيء
1	بند واحد
2	بندان
3	ثلاثة
4	أربعة
5	الخمس جميعها

10. قابلية إعادة الاستخدام (Reusability):

- هل النظام مصمم ليستخدم مجدداً، كمكتبات، components ؟

الدرجة	الوصف
0	لا كود قابل لإعادة الاستخدام.
1	كود قابل لإعادة الاستخدام داخل التطبيق نفسه.
2	أقل من 10% من التطبيق راعى احتياجات أكثر من مستخدم.
3	10% فأكثر راعى احتياجات أكثر من مستخدم.
4	التطبيق مغلّف وموثّق لإعادة الاستخدام مع تخصيص على مستوى الكود.
5	كما السابق مع تخصيص عبر بارامترات يصونها المستخدم.

11. سهولة التركيب (Installation Ease) :

- هل تثبيت النظام بسيط؛ أو يحتاج إعدادات معقدة؟

الدرجة	الوصف
0	لا اعتبارات خاصة، ولا إعداد خاص.
1	لا اعتبارات خاصة لكن يلزم إعداد خاص للتركيب.
2	متطلبات تحويل وتركيب مع أدلة مُختبرة، أثر التحويل غير مهم.
3	كما السابق لكن أثر التحويل مهم.
4	بالإضافة إلى 2: أدوات تركيب وتحويل آلية مُختبرة.
5	بالإضافة إلى 3: أدوات تركيب وتحويل آلية مُختبرة.

12. سهولة التشغيل (Operational Ease) :

- مثلاً هل النظام سهل الإدارة، ويدعم Backup؟

الدرجة	الوصف
0	لا اعتبارات خاصة، فقط إجراء Backup عادي.
1-4	بند أو أكثر من Start-up/Backup/Recovery: يحتاجان تدخّل، أو لا يحتاجان تدخلاً (يُحسب بندين)، تقليل الحاجة لـ Tape mounts ، تقليل الحاجة لمعالجة الورق.
5	تشغيل بدون مراقبة (Unattended) مع استرجاع آلي للأخطاء.

13. مواقع متعدّدة (Multiple Sites) :

- هل النظام يعمل في عدة فروع، وعدة دول، أو عدة بيئات؟

الدرجة	الوصف
0	لا حاجة لمراعاة أكثر من موقع.
1	تصميم لمواقع متعددة على نفس HW/SW.
2	على HW/SW متشابه.
3	على HW/SW مختلف.
4	بالإضافة: توثيق وخطة دعم مُختبرة (الحالة 1 أو 2).
5	بالإضافة: توثيق وخطة دعم مُختبرة (الحالة 3).

14. تسهيل التغيير (Facilitate Change) :

العناصر التي تحسب:

- استعلام/تقارير مرنة لطلبات بسيطة (بند).
- استعلام/تقارير لطلبات متوسطة (يُحسب بندين).
- استعلام/تقارير لطلبات معقدة (يُحسب 3 بنود).
- بيانات تحكم بالأعمال في جداول يديرها المستخدم Online، التغييرات تسري في اليوم التالي.
- نفس السابق لكن التغييرات فورية (يُحسب بندين).

الدرجة	الوصف
0	لا شيء
1-5	عدد البنود المُحققة (1، 2، 3، 4، أو الخمسة كلها).

سادساً : مثال FPA كامل:

نظام متجر الكتروني:

لدينا :

- Inputs=5
- Outputs=3
- Queries=2
- ILF=4
- EIF=1
- نفرض كلها Average

نحسب:

النوع	العدد	الوزن	الناتج
EI	5	4	20
EO	3	5	15
EQ	2	4	8
ILF	4	10	40
EIF	1	7	7

المجموع:

$$UFP=20+15+8+40+7=90$$

إذا كان:

$$TDI=35$$

إذا:

$$VAF=0.65+(0.01*35)=1$$

وبالتالي:

$$AFP=90*1=90$$

سابعاً: مزايا ال FP:

- مستقلة عن لغة البرمجة.
- مفيدة بالتقدير المبكر.
- تساعد بالتسعير.
- معيار هندسي عالمي.
- مناسبة لإدارة المشاريع.

تستخدم في:

- شركات البرمجيات.
- العقود البرمجية.
- تقدير الميزانيات.
- ال freelancing.
- إدارة المشاريع البرمجية.

- تحتاج خبرة عالية.
- بعض التصنيفات Subjective.
- صعوبة بالمشاريع الضخمة.
- تحتاج تحليل دقيق للنظام.

الخاتمة:

تحليل النقاط الوظيفية يعد من أهم الأساليب الهندسية المستخدمة لتقدير حجم البرمجيات بصورة علمية ومنهجية، حيث يركز على الوظائف التي يقدمها النظام للمستخدم بدلاً من التفاصيل التقنية الخاصة بالتنفيذ . ويساعد هذا الأسلوب في تحسين دقة تقدير الوقت والجهد والتكلفة مما يجعله أداة أساسية في إدارة المشاريع البرمجية الحديثة.

Things To Remember :

Tomorrow is a new day.

Focus on the positive things in your life.

Try to see the situation from a different perspective.

Life is too short to waste time on negative thinking.

